

Companion

Личное рабочее пространство разработчика

Документация проекта

Версия 1.0.0

Содержание

- 1. Архитектура приложения
- 2. Структура директорий
- 3. Модели данных
- 4. База данных и миграции
- 5. Слой репозитория
- 6. Сервисный слой
- 7. Презентационный слой
- 8. DI и точка входа
- 9. CI/CD
- 10. Установка и сборка

1. Архитектура приложения

Companion построен на чистой архитектуре (clean architecture) с четырьмя слоями. Каждый слой отвечает за свою зону ответственности (SRP) и зависит только от слоёв ниже (DIP).

- *Core* — инфраструктура: SQLite, константы, исключения, миграции
- *Data* — репозитории: абстрактные интерфейсы и реализации SQLite
- *Domain* — сервисы: бизнес-логика задач и метрик
- *Presentation* — Flutter UI: экраны, виджеты, ChangeNotifier-провайдеры

Управление состоянием — Provider (ChangeNotifier для SettingsService, WorkTimerService, AppStore; Provider для TaskService, MetricsService). Зависимости внедряются через конструктор (ручная DI в main.dart). Принципы SOLID, DRY и KISS соблюдены.

2. Структура директорий

```
lib/
├── main.dart                # Точка входа, DI-контейнер
├── app.dart                 # Корневой виджет, навигация, темы
├── core/                   # Инфраструктура (Core)
│   ├── constants.dart      # Константы, ключи статусов
│   ├── errors/
│   │   └── app_exception.dart # Исключения предметной области
│   ├── database/
│   │   ├── database_helper.dart # Координатор БД (singleton)
│   │   ├── migrations.dart     # Миграции v1 → v4
│   │   └── tables/
│   │       ├── tasks_table.dart
│   │       ├── notes_table.dart
│   │       ├── snippets_table.dart
│   │       ├── activities_table.dart
│   │       ├── work_sessions_table.dart
│   │       └── task_columns_table.dart
│   └── data/               # Слой данных (Data)
│       ├── repositories/
│       │   ├── interfaces/   # Абстракции (DIP)
│       │   │   ├── task_repository.dart
│       │   │   ├── note_repository.dart
│       │   │   ├── snippet_repository.dart
│       │   │   ├── work_session_repository.dart
│       │   │   ├── activity_repository.dart
│       │   │   └── task_column_repository.dart
│       │   └── impl/         # Реализации SQLite
│       │       ├── task_repository_impl.dart
│       │       ├── note_repository_impl.dart
│       │       ├── snippet_repository_impl.dart
│       │       ├── work_session_repository_impl.dart
│       │       ├── activity_repository_impl.dart
│       │       └── task_column_repository_impl.dart
│       └── domain/          # Бизнес-логика (Domain)
│           ├── models/      # Модели данных
│           │   ├── task.dart
│           │   ├── task_column.dart
│           │   ├── note.dart
│           │   ├── snippet.dart
│           │   ├── work_session.dart
│           │   └── activity.dart
│           └── services/
│               ├── task_service.dart # Сервис управления задачами
│               └── metrics_service.dart # Сервис расчёта метрик
├── services/
│   ├── settings_service.dart # ChangeNotifier-сервисы
│   ├── work_timer_service.dart # Настройки (SharedPreferences)
│   ├── app_store.dart        # Таймер рабочего времени
│   └── tray_service.dart     # Хранилище + кеш метрик (JSON)
├── screens/                 # Системный трей
│   ├── dashboard/          # Экраны приложения
│   │   └── dashboard_screen.dart # Дашборд
│   ├── kanban/             # Канбан-доска
│   │   ├── kanban_screen.dart
│   │   ├── dialogs/
│   │   │   ├── task_dialog.dart
│   │   │   └── column_dialog.dart
│   │   └── widgets/
│   │       ├── kanban_board.dart
│   │       ├── kanban_table_tab.dart
│   │       └── archive_tab.dart
│   ├── tasks/              # Детальный просмотр задачи
│   │   └── task_detail_screen.dart
│   ├── notes/              # Заметки
│   │   ├── note_list_screen.dart
│   │   └── note_editor_screen.dart
│   └── snippets/           # Сниметы
```

```
| | | snippet_list_screen.dart
| | | └─ snippet_editor_screen.dart
| | └─ metrics/ # Метрики
| |   └─ metrics_screen.dart
| └─ settings/ # Настройки
|   └─ settings_screen.dart
└─ widgets/ # Переиспользуемые виджеты
    └─ task_card.dart # Карточка задачи
    └─ kanban_column.dart # Колонка канбана
    └─ metric_chart.dart # График-гистограмма
```

3. Модели данных

Task (задача)

Представляет задачу с уникальным номером в формате TASK-XXXX. Статус задаётся строкой (todo, inProgress, done и др.) — это позволяет создавать произвольные колонки в канбане. Поля: id, taskNumber, title, description, status, commitHash, createdAt, updatedAt, completedAt.

TaskColumn (колонка канбана)

Определяет колонку на доске: название, ключ статуса (statusKey), цвет (colorValue в формате ARGB), порядок сортировки, флаг isDefault. При создании БД добавляются 3 колонки по умолчанию: To Do (todo), In Progress (inProgress), Done (done). Пользователь может добавлять и удалять свои колонки.

Note (заметка)

Текстовая заметка с поддержкой Markdown. Может быть привязана к задаче (taskId). Поля: id, title, content, taskId, createdAt, updatedAt.

Snippet (снимет)

Фрагмент кода с подсветкой синтаксиса. Язык выбирается из предустановленного списка. Теги хранятся как JSON-массив в текстовом поле. Может быть привязан к заметке (noteId). Поля: id, title, code, language, tags, noteId, createdAt.

WorkSession (рабочая сессия)

Запись о периоде работы: время старта, время окончания, продолжительность в секундах. Опционально привязана к задаче (taskId). Используется таймером и ручным вводом времени.

Activity (активность)

Событие аудита с типом (enum ActivityType) и временной меткой. Типы: taskCreated, taskCompleted, noteCreated, snippetCreated, timerStarted, timerStopped.

Модель	Поля	Связи
Task	id, taskNumber, title, description, status, commitHash, createdAt, updatedAt, completedAt	
TaskColumn	id, name, statusKey, colorValue, sortOrder, isDefault	—
Note	id, title, content, taskId, createdAt, updatedAt	taskId → Task.id
Snippet	id, title, code, language, tags[], noteId, createdAt	noteId → Note.id
WorkSession	id, startTime, endTime, durationSeconds, taskId	taskId → Task.id
Activity	id, type (enum), timestamp	—

4. База данных и миграции

DatabaseHelper

Синглтон-координатор (`core/database/database_helper.dart`). Инициализирует SQLite через FFI (`sqlite_common_ffi`), открывает файл БД в директории приложения, проводит миграции от текущей версии до актуальной. Предоставляет шесть табличных объектов для CRUD-операций (SRP).

Табличные классы

Каждый класс расположен в `core/database/tables/` и отвечает за операции ровно с одной таблицей базы данных:

Класс	Таблица	Методы
TasksTable	tasks	getAll, getById, insert, update, delete, nextTaskNumber
NotesTable	notes	getAll, insert, update, delete
SnippetsTable	snippets	getAll, insert, update, delete
ActivitiesTable	activities	getAll, getBetween, insert
WorkSessionsTable	work_sessions	getActive, getTodaySessions, getTotalDurationForPeriod, insert, update
TaskColumnsTable	task_columns	getAll, insert, delete

Миграции

Текущая версия схемы БД: 4. Миграции идемпотентны — перед изменением проверяется существование таблиц и колонок через `PRAGMA table_info`.

Версия	Изменения
v1 (initial)	Создание tasks, notes, snippets, activities
v1 → v2	Добавлены taskNumber и completedAt в tasks; создана work_sessions
v2 → v3	Добавлен taskId в work_sessions
v3 → v4	Создана task_columns; вставлены 3 колонки по умолчанию

5. Слой репозитория

Слой репозитория следует принципу DIP (Dependency Inversion): бизнес-логика (сервисы) зависит от абстракций, а не от конкретных реализаций БД. Каждый репозиторий — это пара «интерфейс + реализация».

Интерфейсы (data/repositories/interfaces/)

```
abstract class TaskRepository {
    Future<List<Task>> getAll();
    Future<Task?> getById(String id);
    Future<void> create(Task task);
    Future<void> update(Task task);
    Future<void> delete(String id);
}

abstract class NoteRepository {
    Future<List<Note>> getAll();
    Future<void> create(Note note);
    Future<void> update(Note note);
    Future<void> delete(String id);
}

abstract class SnippetRepository {
    Future<List<Snippet>> getAll();
    Future<void> create(Snippet snippet);
    Future<void> update(Snippet snippet);
    Future<void> delete(String id);
}

abstract class WorkSessionRepository {
    Future<WorkSession?> getActive();
    Future<List<WorkSession>> getTodaySessions();
    Future<int> getTotalDurationForPeriod(DateTime s, DateTime e);
    Future<WorkSession> start({String? taskId});
    Future<void> stop(WorkSession session);
    Future<void> insertManual(WorkSession session);
}

abstract class ActivityRepository {
    Future<List<Activity>> getAll();
    Future<List<Activity>> getBetween(DateTime s, DateTime e);
}

abstract class TaskColumnRepository {
    Future<List<TaskColumn>> getAll();
    Future<void> insert(TaskColumn column);
    Future<void> delete(String id);
}
```

Пример: TaskRepositoryImpl

Реализация принимает DatabaseHelper через конструктор и делегирует операции табличному классу TasksTable. Дополнительно:

- Автоматическая генерация taskNumber через nextTaskNumber()
- Логирование активности: taskCreated при создании, taskCompleted при первом переносе в статус done
- Автоматическая установка/очистка completedAt

6. Сервисный слой

TaskService

Расположение: `domain/services/task_service.dart`. Координирует 4 репозитория (`TaskRepository`, `NoteRepository`, `SnippetRepository`, `TaskColumnRepository`). Содержит бизнес-правила:

- *moveTask* — блокирует перемещение в `InProgress`, если таймер не запущен (`TimerNotRunningException`)
- *deleteColumn* — запрещает удаление стандартных колонок (`CannotDeleteDefaultColumnException`); удаляет все задачи колонки
- *getNoteCounts* / *getSnippetCounts* — подсчёт вложенных сущностей
- *getNotesForTask* / *getSnippetsForTask* — связанные заметки/сниппеты

MetricsService

Расположение: `domain/services/metrics_service.dart`. Агрегирует данные из `ActivityRepository`, `WorkSessionRepository` и `TaskRepository` для расчёта метрик дашборда:

- *getTodayWorkSeconds* — секунды работы сегодня
- *getWeeklyWorkSeconds* — посекундная разбивка по 7 дням
- *getDailyActivityCounts* — активность за 14 дней
- *getTaskStats* — количество всего/в работе/выполнено
- *getTaskCompletedToday* — сколько задач завершено сегодня
- *addManualTime* — вставка ручной записи времени

7. Презентационный слой

Экраны

Экран	Файл	Назначение
Dashboard	dashboard_screen.dart	Сводка: прогресс задач, таймер, график, кнопки создания
Kanban	kanban_screen.dart	Доска / Таблица / Архив с кастомными колонками
Task Detail	tasks/task_detail_screen.dart	Просмотр/редактирование задачи, статус, привязки
Note List	notes/note_list_screen.dart	Список заметок
Note Editor	notes/note_editor_screen.dart	Markdown-редактор с предпросмотром
Snippet List	snippets/snippet_list_screen.dart	Список с поиском и фильтрацией по тегам
Snippet Editor	snippets/snippet_editor_screen.dart	Редактор с подсветкой синтаксиса
Metrics	metrics/metrics_screen.dart	Статистика и графики за 14 дней
Settings	settings/settings_screen.dart	Тема, IDE, язык сниппетов, сброс

Переиспользуемые виджеты

Виджет	Файл	Назначение
TaskCard	task_card.dart	Карточка задачи с цветом статуса, метками заметок и сниппетов
KanbanColumn	kanban_column.dart	DragTarget-колонка с Draggable-карточками
MetricChart	metric_chart.dart	BarChart с тултипами, подписями дней и сводкой

Управление состоянием

Три `ChangeNotifier`-сервиса предоставляются через `MultiProvider` в `main.dart` и подписываются через `context.watch()`:

- `SettingsService` — тема (`light/dark/system`), имя пользователя, язык сниппетов, команда IDE; сохраняется в `SharedPreferences`
- `WorkTimerService` — состояние таймера (`isRunning`, `elapsed`, `activeSession`, `linkedTask`); запускает/останавливает сессии
- `AppStore` — кеш метрик (`todayWorkSeconds`, `weeklyWork`, `dailyActivityCounts`); данные пользователя; сохраняется в `JSON`

`TaskService` и `MetricsService` — простые `Provider` (не `ChangeNotifier`), так как не имеют собственного состояния, требующего подписки. Вызываются через `context.read()`.

8. DI и точка входа

main.dart — порядок инициализации

```
1. WidgetsFlutterBinding.ensureInitialized()
2. windowManager.ensureInitialized()
3. initializeDateFormatting('ru')
4. DatabaseHelper().init()           // SQLite + миграции v4
5. Создание репозиториев (constructor DI):
   TaskRepositoryImpl(db)
   NoteRepositoryImpl(db)
   SnippetRepositoryImpl(db)
   WorkSessionRepositoryImpl(db)
   ActivityRepositoryImpl(db)
   TaskColumnRepositoryImpl(db)
6. Создание сервисов:
   TaskService(taskRepo, noteRepo, snippetRepo, columnRepo)
   MetricsService(activityRepo, workRepo, taskRepo)
7. SettingsService().init()         // SharedPreferences
8. WorkTimerService(workRepo, taskRepo).init()
9. AppStore(workRepo, activityRepo).init()
10. runApp(MultiProvider(           // DI-контейнер
    ChangeNotifierProvider<SettingsService>
    ChangeNotifierProvider<WorkTimerService>
    ChangeNotifierProvider<AppStore>
    Provider<TaskService>
    Provider<MetricsService>
  ))
11. TrayService().init()            // Системный трей + WindowListener
```

Provider-дерево

```
MultiProvider(
  providers: [
    ChangeNotifierProvider<SettingsService>,
    ChangeNotifierProvider<WorkTimerService>,
    ChangeNotifierProvider<AppStore>,
    Provider<TaskService>,
    Provider<MetricsService>,
  ],
  child: CompanionApp(),
)
```

Зависимости создаются ДО runApp и передаются через .value(), что гарантирует их готовность к моменту первого build.

9. CI/CD

GitHub Actions workflow (.github/workflows/build.yml) автоматически собирает release-версии приложения для Windows, Linux и macOS.

Конфигурация

```
name: Build Flutter Desktop
on:
  push: { branches: [main] }
  workflow_dispatch:

jobs:
  build:
    strategy:
      matrix:
        platform:
          - os: windows-latest
            target: windows
            config: --enable-windows-desktop
            build: flutter build windows --release
            artifact: build/windows/x64/runner/Release
          - os: ubuntu-latest
            target: linux
            config: --enable-linux-desktop
            build: flutter build linux --release
            artifact: build/linux/x64/release/bundle
          - os: macos-latest
            target: macos
            config: --enable-macos-desktop
            build: flutter build macos --release
            artifact: build/macos/Build/Products/Release

    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - run: flutter config ${config}
      - run: | # Только для Linux
          sudo apt-get install -y --no-install-recommends \
            clang cmake ninja-build pkg-config libgtk-3-dev liblzma-dev
          if: matrix.os == 'ubuntu-latest'
      - run: flutter pub get
      - run: flutter build ${target} --release
      - run: | # Только для macOS
          cd build/macos/Build/Products/Release
          zip -r ../../../../my_app_macos.zip Runner.app
          if: matrix.os == 'macos-latest'
      - uses: actions/upload-artifact@v4
        with:
          name: my_app_${target}
          path: ${artifact}
          retention-days: 7
```

Детали

- subosito/flutter-action с cache: true — кэширование Flutter SDK и pub-зависимостей
- Linux: предварительная установка clang, cmake, ninja, pkg-config, libgtk-3-dev, liblzma-dev
- macOS: готовый .app пакуется в zip для удобства скачивания
- Windows: Visual Studio C++ tools предустановлены на образе windows-latest
- Артефакты хранятся 7 дней

10. Установка и сборка

Системные требования

- Flutter SDK 3.11 или новее
- *Linux*: clang, cmake, ninja-build, pkg-config, libgtk-3-dev, liblzma-dev
- *Windows*: Visual Studio 2022 с workload «Desktop development with C++»
- *macOS*: Xcode Command Line Tools (xcode-select --install)

Быстрый старт

```
git clone <repo-url>
cd companion
flutter config --enable-linux-desktop
flutter pub get
flutter build linux --release

# Готовый бинарник:
# build/linux/x64/release/bundle/companion
```

Запуск в режиме разработки

```
flutter run -d linux
```

Desktop-интеграция (Linux)

```
# .desktop файл для автозапуска
[Desktop Entry]
Name=Companion
Comment=Личное рабочее пространство разработчика
Exec=/opt/companion/companion
Icon=/opt/companion/assets/icon.png
Terminal=false
Type=Application
Categories=Development;Utility;

# Установка
sudo cp build/linux/x64/release/bundle /opt/companion -r
sudo cp companion.desktop /usr/share/applications/
```

Зависимости Flutter

Пакет	Назначение
provider	Управление состоянием (ChangeNotifier + Provider)
sqlite / sqlite_common_ffi	SQLite для десктопа (через FFI)
path_provider	Путь к директории приложения
shared_preferences	Хранение настроек
intl	Локализация дат и чисел
fl_chart	Графики (BarChart)
flutter_markdown	Рендер Markdown
flutter_highlight + highlight	Подсветка синтаксиса

Пакет	Назначение
uuid	Генерация ID
google_fonts	Шрифт Inter
system_tray	Системный трей
window_manager	Управление окном (close-to-tray)

Конец документации